



RELAZIONE DI PROGETTO

Patavium Open

Corso di Tecnologie Web

Anno Accademico 2025/2026

Componenti del Gruppo

Nominativo	Email Studentesca	Matricola
Marini Anna	<code>anna.marini.7@studenti.unipd.it</code>	2110986
Bellet Eleonora	<code>eleonora.bellet@studenti.unipd.it</code>	2111002
Belluz Diego	<code>diego.belluz@studenti.unipd.it</code>	2043361
Milanese Lorenzo	<code>lorenzo.milanese.1@studenti.unipd.it</code>	2117110

Indirizzo Web del Progetto:

`http://` (*URL pubblico da inserire in seguito*)

Credenziali di Accesso

Amministratore:	<code>admin</code>	Password:	<code>admin</code>
Utente:	<code>user</code>	Password:	<code>user</code>

Indice

1	Origine del progetto	1
1.1	Patavium Open: un torneo fittizio con uno scopo reale	1
1.2	Utenti target	1
2	Architettura del Sito e Scelte Implementative	2
2.1	Struttura Logica e Flusso di Navigazione	2
2.1.1	Area Riservata e Transazionale	3
2.1.2	Area Amministrativa	3
2.2	Pattern Architetturale a Componenti (Template Engine)	3
2.3	Schema di navigazione dell'utente nel sito (metafora della pesca)	3
3	Progettazione del Database	4
3.1	Schema Relazionale	4
3.2	Gestione degli Account	5
3.3	Sistema di Vendita: Programma, Ordini e Biglietti	5
3.4	Informazione e Supporto: News, FAQ e Domande	6
4	Dettagli implementativi	6
4.1	Markup HTML5 e Inclusività (Conformità WCAG 2.1 AA)	6
4.2	Logica di Backend e Gestione dei Dati (PHP)	7
4.2.1	Il Livello dei Controller (php-pages)	7
4.2.2	Il Livello dei Manager (php-manager)	7
4.2.3	Sicurezza Attiva e Validazione (La classe Tool)	7
4.2.4	Accessibilità generata Server-Side e Gestione Errori	8
4.3	Comportamento Client-Side e Interazioni Asincrone (JavaScript)	8
4.4	Sviluppo del Livello di Presentazione (CSS3 e Design System)	9
4.4.1	Design System e CSS Custom Properties	9
4.4.2	Dark Mode Architetturale	9
4.4.3	Accessibilità Visiva e Media Queries Avanzate	10
4.4.4	Approccio DRY e Raggruppamento dei Componenti	10
4.4.5	Ottimizzazione per la Stampa	10
5	Metodologia di Lavoro e Suddivisione dei Compiti	10
5.1	Pianificazione e Organizzazione	10
5.2	Suddivisione dello sviluppo	11
5.2.1	Fase 1: Struttura HTML e Database	11
5.2.2	Fase 2: Logica PHP e JavaScript	11
5.3	Strumenti di comunicazione	11
5.4	Monitoraggio dei Task: La "Task Board" su Google Doc	12

1 Origine del progetto

Il progetto **Patavium Open** nasce dall'esperienza diretta di quattro studenti del corso di Tecnologie Web, accomunati da una passione autentica per il tennis. Nel corso degli ultimi anni, durante la frequentazione di tornei (sia dal vivo che in preparazione di eventuali acquisti di biglietti), ci siamo spesso imbattuti in siti web ufficiali di eventi tennistici che presentavano criticità non trascurabili dal punto di vista dell'usabilità e dell'accessibilità.

In particolare, abbiamo osservato ripetutamente:

- difficoltà nel reperire informazioni chiare su come e dove acquistare i biglietti;
- strutture dei menù poco intuitive, che nascondevano i prezzi, i tipi di abbonamento e i servizi inclusi;
- assenza di un percorso logico che guidasse l'utente — soprattutto il meno esperto — verso la scelta del biglietto o dell'abbonamento più adatto;
- pagine sovraccariche di elementi grafici e testi promozionali, ma povere di dati strutturati (date, campi di gioco, modalità di accesso).

Questi ostacoli, sebbene apparentemente minori, si traducono in una perdita di tempo reale per l'utente finale: ore spese a cercare conferme, a interpretare menù ambigui o a chiedere supporto su forum esterni. Di conseguenza, abbiamo ritenuto utile mettere le nostre competenze al servizio di un progetto dimostrativo che rispondesse a tali lacune.

1.1 Patavium Open: un torneo fittizio con uno scopo reale

Abbiamo quindi immaginato un torneo di tennis internazionale che si svolgesse idealmente a Padova, battezzandolo **Patavium Open** (dal nome latino della città, *Patavium*). L'obiettivo non era realizzare il sito di un evento esistente, ma **progettare e sviluppare un prototipo web che risolvesse in modo concreto i problemi di chiarezza, accessibilità e reperibilità dei contenuti** riscontrati nei siti reali.

Il sito del Patavium Open si propone quindi di:

- esporre in modo esplicito e gerarchico tutte le informazioni sui biglietti e sugli abbonamenti;
- guidare l'utente nell'acquisto con pochi passaggi ben identificabili;
- garantire l'accesso ai contenuti anche a persone con disabilità o con dispositivi e connessioni limitate;
- separare nettamente contenuto, presentazione e comportamento (HTML, CSS, PHP/JS) come richiesto dalle specifiche del corso.

1.2 Utenti target

L'utenza primaria è costituita da **appassionati di tennis** di qualsiasi età e competenza tecnologica: dai giovani che seguono il circuito ATP/WTB, ai genitori che accompagnano ragazzi ai tornei, fino agli spettatori occasionali. Tutti questi utenti condividono l'esigenza di:

- trovare rapidamente le date e il luogo del torneo;
- confrontare le opzioni di biglietteria (giornaliero, settimanale, tribuna centrale, campo secondario, eventuali pacchetti famiglia);
- completare l'acquisto con un flusso controllato (convalida lato client e lato server, salvataggio in database).

Abbiamo scelto di non includere funzionalità “di facciata” (like, commenti, social network) se non strettamente necessarie, per concentrarci invece sugli aspetti che generano maggiore frustrazione nei siti esistenti: **chiarezza e controllabilità dell’interazione**.

Nella progettazione abbiamo tenuto conto anche di possibili motori di ricerca (SEO): il sito deve rispondere a query tipiche come “*Patavium Open biglietti*”, “*quando si gioca il torneo di Padova*”, “*prezzi abbonamenti tennis Padova*”. Le azioni intraprese per migliorare il ranking vengono discusse nella sezione dedicata dell’analisi.

2 Architettura del Sito e Scelte Implementative

L’architettura del portale *Patavium Open* è stata concepita per guidare l’utente attraverso un flusso di navigazione intuitivo, adottando un approccio modulare. Il sistema separa in modo netto la logica di business dall’interfaccia utente, organizzando le risorse in directory specializzate e standardizzando i nomi dei file secondo la convenzione `snake_case` per garantire pulizia e compatibilità.

2.1 Struttura Logica e Flusso di Navigazione

Area Pubblica e Informativa

Accessibile a tutti, è la vetrina principale del portale e immerge subito l’utente nel contesto del tennis professionistico:

- **Home Page (`pages/index.html`):** Il fulcro del sito. Offre una panoramica immediata sull’evento, le notizie in evidenza e un accesso rapido alle azioni principali (come l’acquisto dei biglietti).
- **Sezione Biglietteria (`php-pages/biglietti.php`):** Presentata tramite card chiare e descrittive, divide l’offerta in tre opzioni pensate per le reali esigenze del pubblico:
 - *Abbonamento (`php-pages/abbonamento.php`):* Accesso completo per l’intera durata del torneo.
 - *Ground Pass (`php-pages/ground_passes.php`):* Titolo d’ingresso per vivere l’atmosfera del parco sportivo, accedere alle aree ristoro e seguire i match dai maxischermi, senza un posto numerato negli stadi principali.
 - *Singola Sessione (`php-pages/single_session.php`):* Acquisto flessibile e mirato per la sessione diurna o serale di una specifica giornata.
- **News (`php-pages/news.php`):** Modulo dinamico per gli aggiornamenti, i risultati in tempo reale e gli approfondimenti. Mantiene vivo l’interesse del pubblico e favorisce un’ottima indicizzazione sui motori di ricerca.
- **Supporto Interattivo - FAQ (`php-pages/faq.php`):** Molto più di un semplice elenco statico. Se un utente non trova la risposta che cerca tra le domande frequenti, può inviare un quesito personalizzato. Questo crea un vero sistema di ticketing: la domanda arriva alla dashboard dell’amministratore, che può rispondere facendo apparire la notifica direttamente nell’area personale dell’utente.
- **Informativa Legale (`php-pages/termini.php`, `php-pages/privacy.php`):** Pagine dedicate alla trasparenza, logicamente separate dal supporto, dove consultare i termini di servizio e le policy sulla privacy.

2.1.1 Area Riservata e Transazionale

Le funzionalità interattive e legate agli acquisti sono protette da autenticazione:

- **Autenticazione (`php-pages/login.php`, `php-pages/registrazione.php`):** Pagine che gestiscono l'accesso e la creazione sicura degli account.
- **Carrello (`php-pages/carrello.php`):** Il cuore delle transazioni. L'utente gestisce i biglietti scelti, visualizza il riepilogo dinamico dei costi e conclude l'acquisto.
- **Area Utente (`php-pages/area_utente.php`):** La dashboard personale. Qui l'utente ritrova lo storico dei suoi ordini, i ticket generati e un centro notifiche dove arrivano le risposte dell'amministrazione ai quesiti posti nella sezione FAQ.

2.1.2 Area Amministrativa

- **Gestione Torneo (`php-pages/area_admin.php`):** Pannello di controllo riservato allo staff. Permette di gestire attivamente i contenuti: pubblicare nuove notizie, aggiungere FAQ generali e rispondere ai ticket di supporto aperti dagli utenti, garantendo che il sito sia sempre aggiornato e interattivo.

2.2 Pattern Architetturale a Componenti (Template Engine)

Per rispettare il principio DRY (Don't Repeat Yourself) ed evitare codice ridondante, l'interfaccia non è stata costruita con file monolitici, ma tramite un motore di template custom lato server gestito dal PHP. Il sistema sfrutta due cartelle principali:

- **La directory `pages/`:** Contiene i layout macroscopici. Questi file HTML definiscono la struttura di base e posizionano dei segnaposto (come `[Header]` o `[lista_ordini]`) in attesa dei dati.
- **La directory `item/`:** Contiene i *Template Fragments*. Elementi grafici che si ripetono (come la card di un biglietto, la riga di una notifica o il blocco di una notizia) sono stati isolati qui in singoli file HTML.

Grazie a questa architettura, quando bisogna mostrare una lista di notizie, il PHP recupera i dati dal database, riempie il frammento HTML prelevato da `item/` per ogni singola news, e inietta il risultato finale nella pagina. Se in futuro vorremo modificare il design di una card, basterà aggiornare un solo file per vederlo cambiato in tutto il sito.

2.3 Schema di navigazione dell'utente nel sito (metafora della pesca)

Il portale web del Patavium Open presenta un'architettura dell'informazione solida e strutturata, concepita per assecondare i molteplici comportamenti di ricerca tipici degli utenti. Possiamo analizzare l'attuale configurazione del sito attraverso la "metafora della pesca":

- **Tiro perfetto:** In questo scenario l'utente sa esattamente quello che sta cercando. Il portale asseconda questa necessità attraverso un menu di navigazione privo di ambiguità. Un'attenta analisi sulle intenzioni d'uso ha evidenziato che la maggioranza dell'utenza accede al sito con l'obiettivo primario di acquistare i biglietti; per questo motivo si è scelto di omettere la tradizionale barra di ricerca (ritenuta in questo contesto dispersiva) a favore di un approccio molto più diretto: un grande pulsante "Acquista i tuoi biglietti" campeggia subito nella Home Page. L'utente è così messo nelle condizioni di compiere il suo "tiro perfetto" verso la conversione nel minor tempo possibile.

- **Trappola per aragoste:** La struttura si adatta perfettamente anche a chi naviga senza un'idea predefinita, attirandolo e guidandolo progressivamente. Il flusso di acquisto nella sezione Biglietti ne è la dimostrazione pratica: l'utente entra con un'intenzione generica e viene incanalato, come in una trappola a cilindri concentrici, verso scelte sempre più specifiche (seleziona prima la macro-tipologia tra Abbonamento, Ground Passes o Singola Sessione, per poi scendere nel dettaglio del posto e della giornata).
- **Pesca con la rete:** L'esigenza di esplorare un argomento in tutta la sua totalità è soddisfatta dalla sezione News. Questo ambiente è progettato per gli utenti che non vogliono perdersi nulla, offrendo una panoramica aggregata di tutti gli aggiornamenti e i risultati, permettendo di "catturare" con un'unica mossa tutta l'attualità legata al torneo.
- **Boa di segnalazione:** Per supportare la necessità di salvare e recuperare informazioni utili in un secondo momento, il portale integra strumenti strategici. Il Carrello funge da memoria temporanea per mantenere traccia dei biglietti in fase di selezione, mentre l'Area Personale rappresenta la vera e propria "boa di segnalazione", offrendo all'utente uno spazio privato e sicuro dove recuperare in qualsiasi momento i propri titoli d'accesso, lo storico ordini e le comunicazioni dallo staff.

3 Progettazione del Database

Per la gestione dei dati del portale abbiamo optato per un database relazionale (MySQL/MariaDB), progettato partendo dalla definizione del modello Entità-Relazione (ER). L'obiettivo principale è stato mantenere il database normalizzato, evitando la ridondanza delle informazioni e garantendo l'integrità dei dati tramite l'uso rigoroso delle chiavi esterne (*Foreign Key*).

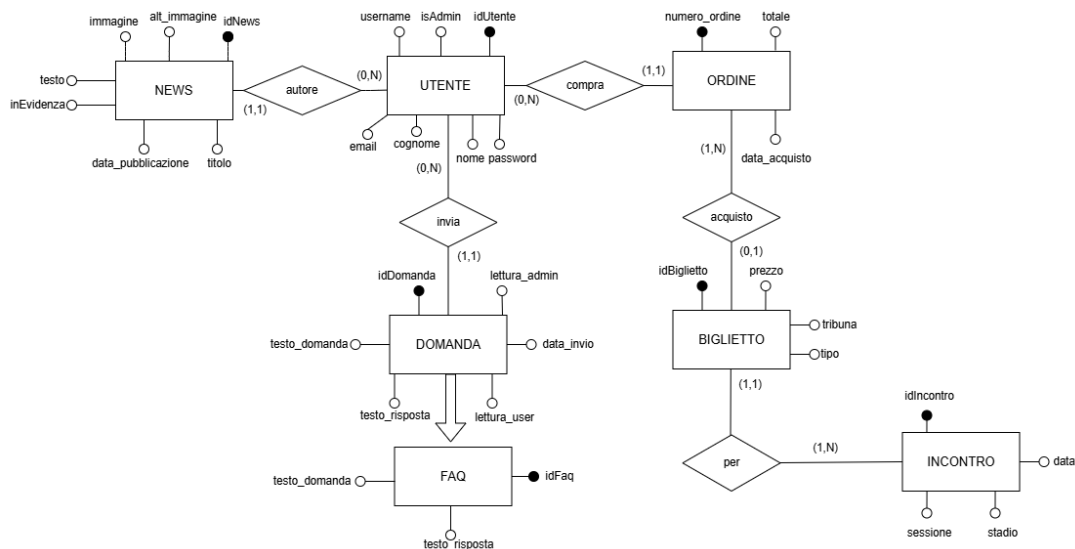


Figure 1: Schema E-R del database Patavium Open

3.1 Schema Relazionale

Di seguito è riportato lo schema logico relazionale derivato dal modello concettuale. Le relazioni presentano le chiavi primarie indicate con la sottolineatura, mentre gli attributi opzionali, che ammettono il valore NULL, sono contrassegnati da un asterisco (*).

- **UTENTE**(idUtente, username, email, password, nome, cognome, isAdmin)
- **INCONTRO**(idIncontro, data, sessione*, stadio)

- **NEWS**(idNews, titolo, testo, data_pubblicazione, immagine, alt_immagine, idAutore, inEvidenza)
FK: idAutore → **UTENTE**(idUtente)
- **DOMANDA**(idDomanda, testo_domanda, testo_risposta*, lettura_admin, lettura_user, data_invio, idUtente)
FK: idUtente → **UTENTE**(idUtente)
- **ORDINE**(numero_ordine, totale, data_acquisto, idUtente)
FK: idUtente → **UTENTE**(idUtente)
- **BIGLIETTO**(idBiglietto, prezzo, tribuna*, tipo*, numero_ordine*, idIncontro)
FK: numero_ordine → **ORDINE**(numero_ordine)
FK: idIncontro → **INCONTRO**(idIncontro)
- **FAQ**(idFaq, testo_domanda, testo_risposta)

Come si può notare dallo schema in Figura 1, il database è strutturato in tre macro-aree logiche: la gestione degli account, l'area di e-commerce (ordini e biglietti) e l'area di interazione e supporto (news e ticket).

3.2 Gestione degli Account

Il cuore del sistema è la tabella **UTENTE**. Al suo interno salviamo i dati anagrafici e le credenziali di accesso. Per questioni di sicurezza, le password non vengono mai salvate in chiaro nel database, ma sono cifrate utilizzando la funzione nativa di hashing `bcrypt` di PHP.

Invece di creare due tabelle separate per i clienti e per gli amministratori, abbiamo preferito semplificare la struttura inserendo un campo booleano chiamato `isAdmin`. Se questo flag è a 1, il sistema riconosce l'utente come membro dello staff e gli sblocca l'accesso al pannello di controllo (`area_admin.php`); altrimenti, l'utente naviga con i privilegi standard.

3.3 Sistema di Vendita: Programma, Ordini e Biglietti

La gestione della biglietteria è stata progettata per prevenire alla radice problemi di concorrenza o di *overbooking* (ad esempio, quando due utenti tentano di comprare lo stesso posto nello stesso istante). Per farlo, utilizziamo tre tabelle interconnesse:

- **INCONTRO**: Contiene il calendario del torneo. Ogni riga rappresenta un evento, definendone la data, lo stadio (es. Patavium Arena o Giotto Court) e il tipo di sessione (diurna, serale o ground).
- **BIGLIETTO**: La nostra scelta implementativa è stata quella di "pre-generare" fisicamente nel database tutti i biglietti disponibili per ogni partita, ancor prima che vengano venduti. Ogni biglietto nasce con un prezzo, una tribuna e il collegamento all'incontro, ma con il campo `numero_ordine` impostato a `NULL`.
- **ORDINE**: Quando un utente completa un acquisto dal carrello, il sistema crea un record in questa tabella (registrando il totale pagato e la data) e va semplicemente a inserire l'ID di questo nuovo ordine dentro i biglietti scelti. Se un biglietto ha il `numero_ordine` valorizzato, il sistema sa che è già stato venduto e non lo mostra più tra quelli disponibili.

3.4 Informazione e Supporto: News, FAQ e Domande

L'interazione con gli utenti e l'aggiornamento del portale si basano su tre tabelle specifiche:

- **NEWS:** Tabella dedicata agli articoli e agli aggiornamenti. È collegata all'utente amministratore che l'ha scritta (tramite `idAutore`) e include i campi per il percorso dell'immagine e per il suo testo alternativo (`alt_image`), fondamentale per l'accessibilità. Il flag `inEvidenza` è molto utile per permettere all'admin di scegliere quali notizie far comparire in primo piano nella Home Page.
- **FAQ:** Una tabella indipendente e di consultazione pubblica, che contiene semplicemente le domande frequenti e le relative risposte.
- **DOMANDA:** Rappresenta il vero e proprio sistema di supporto clienti (Ticketing). Qualsiasi utente registrato può inviare un quesito. Al momento dell'invio, il sistema compila il `testo_domanda` e lascia temporaneamente vuoto il `testo_risposta`. Per gestire le notifiche in modo efficiente, abbiamo inserito due flag: `lettura_admin` e `lettura_user`. Quando l'utente scrive, il sistema avvisa l'amministratore; non appena l'admin risponde, il flag dell'utente scatta, facendogli comparire la notifica visiva direttamente all'interno della sua Area Personale, in modo che sappia di aver ricevuto assistenza.

4 Dettagli implementativi

4.1 Markup HTML5 e Inclusività (Conformità WCAG 2.1 AA)

Particolare attenzione è stata posta alla stesura del codice HTML, considerato non come semplice strumento di impaginazione, ma come livello semantico fondamentale per l'accessibilità e l'indicizzazione. Le interfacce sono state sviluppate rispettando rigorosamente le linee guida WCAG 2.1 livello AA, attraverso l'adozione dei seguenti pattern:

- **HTML5 Strict e Graceful Degradation:** Il markup è stato scritto in rigoroso HTML5, prestando attenzione a mantenere una sintassi di tipo XML (chiusura esplicita di tutti i tag e attributi correttamente quotati). Il sito è stato progettato per "degradare con grazia": anche in assenza di fogli di stile o con JavaScript disabilitato, il contenuto rimane perfettamente leggibile, gerarchico e navigabile grazie al solo HTML puro.
- **Landmark Semantici e Segmentazione:** La struttura di ogni pagina evita l'abuso di contenitori generici (`<div>`), affidandosi a tag semantici nativi (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`). Questo partizionamento logico permette agli screen reader (come NVDA o JAWS) di generare un albero di navigazione corretto, consentendo agli utenti con disabilità visive di orientarsi immediatamente all'interno dei contenuti.
- **Liste di Definizione Semantiche:** Per la presentazione di dati strutturati (come i dettagli dei match o le specifiche dei biglietti), si è optato per l'utilizzo delle *Definition Lists* (`<dl>`, `<dt>`, `<dd>`). Questa scelta architetturale crea un legame logico indissolubile tra l'etichetta (es. "Data:") e il suo valore, garantendo una gerarchia informativa impeccabile e semanticamente superiore rispetto a semplici paragrafi affiancati.
- **Feedback Asincroni e Live Regions:** Per garantire un'esperienza utente reattiva e accessibile, la validazione dei form (es. Login e Registrazione) fa uso delle *ARIA Live Regions*. Implementando l'attributo `aria-live="polite"`, il sistema annuncia in tempo reale i messaggi di errore o di successo, assicurando che l'utente venga notificato in modo vocale senza subire disorientanti ricaricamenti di pagina o perdite di focus.

- **Skip-Link e Gestione del Focus:** Per favorire la navigazione tramite tastiera, è stato integrato uno *Skip Link* ("Vai al contenuto") visibile solo al momento della ricezione del focus. Inoltre, tutti gli elementi interattivi presentano indicatori visivi di focus (*outline*) studiati per mantenere un elevato rapporto di contrasto sia nel tema chiaro che nella Dark Mode.
- **Contenuti Multimediali e Decorativi:** Le immagini informative sono provviste di attributi `alt` concisi. Al contrario, gli elementi puramente estetici (come file vettoriali SVG o icone di accompagnamento) sono stati silenziati tramite gli attributi `alt=""` e `aria-hidden="true"`, al fine di non generare "rumore cognitivo" durante la lettura assistita.
- **Utilità per Screen Reader:** Per menu a tendina o pannelli espandibili sono stati usati i tag nativi `<details>` e `<summary>`, che garantiscono nativamente l'accessibilità da tastiera. Infine, informazioni visivamente superflue ma necessarie al contesto sono state preservate per gli screen reader utilizzando la classe di utilità CSS `.sr-only` (che nasconde l'elemento a schermo senza rimuoverlo dal DOM) e l'attributo `lang="en"` per evitare errori di pronuncia sui termini inglesi.

4.2 Logica di Backend e Gestione dei Dati (PHP)

Il cuore del sistema è stato sviluppato in PHP puro, adottando un pattern architetturale molto vicino al modello MVC (Model-View-Controller). L'obiettivo primario è stato separare la gestione delle richieste HTTP dalla logica di interazione con il database. Per farlo, abbiamo diviso gli script in due macro-directory: `php-pages` e `php-manager`.

4.2.1 Il Livello dei Controller (`php-pages`)

I file contenuti in questa cartella (come `abbonamento.php` o `area_admin.php`) fungono da orchestratori. Il loro compito è intercettare le richieste dell'utente (tramite GET o POST), verificare in modo stringente i permessi di sessione e dialogare con i Manager per ottenere i dati necessari. Una volta recepite le informazioni, il controller sfrutta il nostro motore di template custom per iniettarle all'interno dei file HTML, per poi restituire la pagina finita al browser. Abbiamo inoltre gestito le chiamate asincrone (AJAX), permettendo ai controller di interrompere l'esecuzione e restituire risposte pulite in formato JSON per le interazioni dinamiche dell'interfaccia.

4.2.2 Il Livello dei Manager (`php-manager`)

Tutta la logica di business e l'interazione con il database relazionale sono state isolate nei file Manager. Classi come `TicketManager` e `NewsManager` incapsulano interamente le query SQL. Abbiamo dedicato molta cura alla stesura di query complesse e ottimizzate: ad esempio, il calcolo della disponibilità reale degli abbonamenti non è un dato statico, ma viene elaborato al volo incrociando i biglietti invenduti con i raggruppamenti delle 14 sessioni del torneo. Questo isolamento garantisce che un'eventuale futura modifica alla struttura del database richiederà aggiornamenti solo all'interno dei Manager, lasciando intatto il resto del sito.

4.2.3 Sicurezza Attiva e Validazione (La classe `Tool`)

La sicurezza è stata trattata come requisito fondamentale e non come aggiunta a posteriori. Abbiamo centralizzato tutte le funzioni di utilità e validazione nella classe statica `Tool`:

- **Prevenzione XSS (Cross-Site Scripting):** Qualsiasi input inserito dall'utente viene ripulito tramite il metodo `Tool::pulisciInput()`. Questa funzione combina `strip_tags` e `htmlspecialchars` con i flag più restrittivi (`ENT_QUOTES | ENT_HTML5`), garantendo

che codice malevolo o script iniettati nei form vengano neutralizzati prima di arrivare al database.

- **Prevenzione SQL Injection:** Utilizziamo ovunque i **Prepared Statements** (`bind_param`), delegando al driver del database il compito di sanificare i parametri, rendendo le iniezioni SQL strutturalmente impossibili.
- **Validazione Upload Sicura:** Nell'area amministrativa, il sistema di caricamento delle immagini per le notizie non si limita a controllare l'estensione del file (es. `.jpg` o `.webp`). Sfruttiamo controlli MIME avanzati tramite `getimagesize()` per verificare che il file sia realmente un'immagine e non uno script PHP mascherato, bloccando alla radice potenziali intrusioni nel server.

4.2.4 Accessibilità generata Server-Side e Gestione Errori

Abbiamo integrato l'accessibilità direttamente nel motore PHP, evitando di delegare la modifica degli stati visivi solo a JavaScript. Il backend calcola e inietta gli attributi ARIA appropriati; ad esempio, se le query di `TicketManager` riportano che i posti per una tribuna sono esauriti, il file `abbonamento.php` stampa dinamicamente l'attributo `aria-disabled="true"` e le classi di blocco direttamente nel DOM.

Infine, abbiamo prestato molta attenzione alla gestione degli errori e alle operazioni critiche sul database. I passaggi più delicati, come il salvataggio di un ordine dal carrello, sono inseriti all'interno di blocchi `try-catch`. Se qualcosa va storto durante il checkout, blocchiamo l'operazione facendo un *rollback*, così evitiamo di salvare dati incompleti o corrotti. L'utente viene poi reindirizzato a una pagina sicura con un messaggio di errore chiaro (generato tramite `Tool::buildMessage()`). In questo modo evitiamo che la pagina crashi mostrando i classici errori nativi di PHP, che finirebbero per svelare percorsi del server o dettagli delle query SQL. Con la stessa logica di sicurezza abbiamo gestito il logout: non ci limitiamo a svuotare l'array di sessione sul server, ma andiamo a cancellare fisicamente i cookie dal browser del client, azzerando qualsiasi rischio di riutilizzo della sessione.

4.3 Comportamento Client-Side e Interazioni Asincrone (JavaScript)

Il livello di interazione lato client è stato sviluppato interamente in *Vanilla JavaScript* (ECMAScript 6+), escludendo l'uso di librerie esterne o framework. L'obiettivo è stato quello di arricchire l'esperienza utente mantenendo il caricamento delle pagine rapido e il codice manutenibile.

L'architettura JavaScript del progetto si fonda su quattro pilastri:

- **Comunicazione Asincrona (AJAX e Fetch API):** Per evitare ricaricamenti di pagina durante le operazioni di routine, abbiamo deciso di utilizzare chiamate asincrone tramite la `Fetch API`. Un esempio è la rimozione di un biglietto nel `carrello.js`: quando l'utente elimina un elemento, il client invia una richiesta POST al server con l'indice del biglietto; alla ricezione della conferma (in formato JSON), il JavaScript aggiorna il totale a schermo e rimuove l'elemento dal DOM applicando un'animazione CSS, senza interrompere la navigazione.
- **Gestione Dinamica delle Viste:** In sezioni complesse come l'Area Admin o l'Area Utente, invece di avere file HTML separati per visualizzare la lista delle FAQ o il form per inserirne una nuova, il JavaScript intercetta i click sui menu laterali e "accende o spegne" le diverse sezioni della pagina manipolando le classi CSS (`.active` o `.hidden`). Questo approccio azzerava i tempi di caricamento durante la navigazione interna dei pannelli di controllo.

- **Controllo dei form in tempo reale e gestione errori:** Nel file `validazione-reg.js` volevamo dare un aiuto immediato a chi si registra, senza aspettare che preme il tasto "Invia". Mentre l'utente scrive la password, usiamo delle espressioni regolari per verificare subito i requisiti (lettere maiuscole, numeri, caratteri speciali) e aggiorniamo una checklist a schermo che si colora man mano che le regole vengono rispettate. Abbiamo curato anche l'accessibilità dei messaggi d'errore asincroni (le chiamate AJAX): quando qualcosa va storto, il JavaScript crea al volo un blocco con l'attributo `aria-live="assertive"`. In questo modo forziamo lo screen reader ad annunciare subito il problema, avvisando tempestivamente l'utente.
- **Gestione del menu da tastiera:** Nello script `nav.js` abbiamo curato l'accessibilità del menu "hamburger" per mobile. Abbiamo aggiunto un ascoltatore per l'evento `keydown` in modo che, se l'utente preme il tasto **Escape** con la tendina aperta, il menu si chiude in automatico e l'attributo `aria-expanded` torna a `false`. La cosa più importante che abbiamo implementato è il ritorno del *focus*: una volta chiuso il menu, il cursore torna esattamente sul pulsante di apertura. In questo modo, chi naviga solo con la tastiera non rischia di ritrovarsi sbalzato all'inizio o in fondo alla pagina.
- **Dark Mode e salvataggio delle preferenze:** Nel file `theme.js` abbiamo gestito il tema scuro evitando di manipolare gli stili inline tramite JavaScript. Lo script si limita ad applicare l'attributo `data-theme="dark"` direttamente al tag `<html>`, lasciando al CSS il compito di scambiare le variabili colore. La scelta dell'utente viene salvata nel `localStorage` per mantenerla attiva durante tutta la navigazione. Come accortezza in più, per i nuovi visitatori, utilizziamo `window.matchMedia` per leggere le impostazioni del sistema operativo (ad esempio, se il telefono o il PC dell'utente è già in modalità scura) e adattiamo il sito di conseguenza fin dal primo caricamento.

4.4 Sviluppo del Livello di Presentazione (CSS3 e Design System)

Il foglio di stile del progetto è stato sviluppato adottando un approccio architetturale solido e moderno, senza l'ausilio di framework esterni, per garantire il massimo controllo sulle performance e sulla personalizzazione visiva.

4.4.1 Design System e CSS Custom Properties

L'intera interfaccia si basa su un Design System centralizzato governato da CSS Custom Properties (variabili) dichiarate nella pseudo-classe `:root`. Questa scelta ha permesso di tokenizzare l'intera palette cromatica (suddivisa in colori di superficie, testo e accenti), le spaziature fluide e i livelli di opacità precompilati (es. `-col-shadow-15`). L'utilizzo dei token ha garantito una totale coerenza visiva lungo tutte le macro-sezioni dell'applicativo, facilitando future operazioni di manutenzione.

4.4.2 Dark Mode Architetturale

L'implementazione del tema scuro (*Dark Mode*) è stata progettata per essere scalabile e priva di ridondanze strutturali. Invece di riscrivere le regole CSS per ogni singolo componente, il tema scuro viene attivato a livello globale mediante l'attributo `[data-theme="dark"]`, il quale si limita a sovrascrivere i valori delle variabili radice (come `-col-bg`, `-col-surface` e le palette dei testi). Di conseguenza, tutti i moduli ereditano dinamicamente i nuovi contrasti, garantendo un'esperienza immersiva al buio senza incrementare il debito tecnico o il peso del file.

4.4.3 Accessibilità Visiva e Media Queries Avanzate

In totale conformità con le direttive WCAG, un’attenzione particolare è stata posta all’inclusività motoria e visiva del front-end:

- **Gestione del Focus:** Tutti gli elementi interattivi (pulsanti, link, controlli dei form) presentano stati di `:focus-visible` rigorosamente standardizzati, i quali applicano un `outline` marcato e spaziato per guidare agevolmente la navigazione da tastiera.
- **Animazioni Ridotte (`prefers-reduced-motion`):** Sono state integrate speciali *media queries* in grado di interrogare le preferenze di sistema dell’utente. Qualora l’utente abbia richiesto una riduzione del movimento (per prevenire disagi vestibolari), il CSS disabilita nativamente tutte le transizioni, le animazioni e lo scorrimento fluido (`scroll-behavior: auto`).

4.4.4 Approccio DRY e Raggruppamento dei Componenti

Per ottimizzare la pulizia del codice e scongiurare le ripetizioni (rispettando il pattern architetturale DRY - *Don't Repeat Yourself*), le regole strutturali condivise sono state consolidate in macro-selettori di raggruppamento. Le proprietà grafiche identiche — come `border-radius`, transizioni e `box-shadow` delle *card* o i *padding* e i colori di sfondo dei pulsanti — sono centralizzate. Le classi specifiche di ciascun componente si limitano a definire unicamente le logiche interne di layout (tramite *CSS Grid* e *Flexbox*), mantenendo un’architettura a componenti estremamente robusta.

4.4.5 Ottimizzazione per la Stampa

Il livello di presentazione è stato arricchito con regole mirate per la corretta impaginazione e la fruizione su supporti cartacei. Il layout di stampa agisce in modo sottrattivo e normalizzante:

- Gli elementi interattivi non essenziali per la lettura (come il menu di navigazione, i pulsanti d’acquisto, la sidebar utente e lo *skip-link*) vengono nascosti automaticamente tramite la direttiva `display: none`.
- Il foglio di stile neutralizza le ombreggiature e disabilita i background colorati, forzando un alto contrasto basato su testo nero puro (`var(-nero-puro)`) su sfondi bianchi (`var(-bianco-puro)`).
- Per prevenire il troncamento dei testi a metà tra due pagine stampate, i contenitori principali (come i riquadri delle date, i biglietti e gli elementi *FAQ*) fanno uso delle direttive `page-break-inside: avoid` e `break-inside: avoid`.
- I collegamenti ipertestuali sono stati espansi tramite l’uso degli pseudo-elementi (`::after` e `attr(href)`), permettendo di stampare fisicamente l’URL di destinazione accanto al testo del link.

5 Metodologia di Lavoro e Suddivisione dei Compiti

Il lavoro di sviluppo è stato portato avanti dal nostro gruppo composto da quattro persone: Anna Marini, Eleonora Bellet, Lorenzo Milanese e Diego Belluz.

5.1 Pianificazione e Organizzazione

Data l’importanza di prendere decisioni condivise fin da subito, le fasi iniziali di *brainstorming* e la definizione dello schema ER (il modello del database) si sono svolte tramite sessioni di confronto su Discord. Questa scelta è stata cruciale perché ci ha permesso di decidere insieme

la struttura del database e l'architettura generale del sito prima di scrivere anche una sola riga di codice.

Durante questi incontri abbiamo anche definito delle regole comuni di base per non "incasinarci" durante lo sviluppo:

- **Naming Convention:** abbiamo deciso di usare esclusivamente il formato `snake_case` per i nomi di tutti i file.
- **Struttura delle cartelle:** abbiamo creato una suddivisione chiara tra i file PHP e i file HTML per tenere tutto in ordine.

5.2 Suddivisione dello sviluppo

Per lavorare in modo efficiente e parallelo, abbiamo deciso di non metterci mai a lavorare sugli stessi file contemporaneamente, evitando così conflitti e sovrapposizioni.

5.2.1 Fase 1: Struttura HTML e Database

Mentre tre di noi (Eleonora, Diego e Anna) si dedicavano alla costruzione dell'HTML, Lorenzo si è occupato del *backend* inteso come base dati, creando il file `patavium_open.sql` e gestendo la logica del database. Gli altri tre componenti hanno suddiviso le pagine del sito: chi si occupava della sezione "Biglietti" ha gestito sia la pagina principale che la creazione delle *card* e degli *item* specifici (come le sessioni e i Ground Pass), così da mantenere omogeneo lo stile di ogni singola pagina.

5.2.2 Fase 2: Logica PHP e JavaScript

Una volta completata la parte strutturale, ci siamo spostati sulla logica vera e propria. Anche in questa fase abbiamo lavorato in parallelo: ognuno di noi ha preso in carico dei moduli specifici. Ad esempio, chi aveva già sviluppato la pagina Biglietti ha curato anche la logica di inserimento nel carrello, mentre altri si sono concentrati sulla gestione delle FAQ o sull'Area Utente.

Per quanto riguarda JavaScript e PHP, abbiamo evitato di scrivere codice sparso: le funzioni più complesse o riutilizzabili sono state create da chi di noi era più focalizzato su quella specifica funzionalità, per poi essere condivise e richiamate da tutti. Questo ci ha permesso di non dover fare ognuno il "proprio" PHP o il "proprio" JS, ma di avere un codice che funzionasse in modo uguale su tutto il sito.

Questo approccio ci ha permesso di procedere in parallelo, completando le varie aree funzionali (come il carrello, l'area admin e il sistema di notifiche) in tempi brevi, riuscendo a integrare tutto correttamente grazie al continuo allineamento durante le nostre sessioni di lavoro.

5.3 Strumenti di comunicazione

Per mantenere il team sempre allineato e gestire le diverse fasi di sviluppo in modo ordinato, abbiamo adottato una strategia basata su strumenti di comunicazione e collaborazione immediati, evitando complessità eccessive che avrebbero potuto rallentare il lavoro.

La comunicazione quotidiana si è basata principalmente su due pilastri:

- **Discord:** utilizzato per le chiamate di gruppo e per le sessioni di brainstorming più lunghe. È stato lo spazio virtuale in cui abbiamo preso le decisioni più importanti, come la definizione del database e la struttura delle cartelle.
- **WhatsApp:** utilizzato per scambi di informazioni rapide, alert immediati e per mantenere un contatto costante e informale durante le ore di lavoro individuale.

5.4 Monitoraggio dei Task: La “Task Board” su Google Doc

Per tracciare lo stato di avanzamento del lavoro, abbiamo creato un documento Google condiviso contenente una tabella di gestione dei task. L’idea di fondo era quella di avere uno strumento simile a un sistema di *Issue Tracking* (come quello di GitHub), ma più semplice e immediato da consultare per tutti i membri del gruppo.

La tabella era strutturata in modo molto chiaro:

- **Obiettivo:** una breve descrizione del task (es. “Creazione pagina FAQ”, “Validazione form Login”).
- **Responsabile:** il nome del componente del team incaricato di completare il compito.
- **Stato:** un campo dinamico che aggiornavamo costantemente con tre stati: *Da fare*, *In corso*, *Completato*.

Questa “bacheca” digitale ci ha permesso di avere sempre una visione d’insieme chiara di cosa mancava, chi stava lavorando su cosa e a che punto era il progetto. È stata la nostra soluzione per evitare che due persone lavorassero sullo stesso task senza saperlo e per assicurarci di arrivare alla consegna con tutti i pezzi pronti e testati.